

Mobile App Developer

Step 5: Build Real-world Apps

CareerCompass • Detailed Concept Notes • Research-backed learning guide

Concept & Learning Objective

A portfolio application becomes valuable when it demonstrates engineering decisions rather than tutorial completion. Real apps require requirements, state management, persistence, validation, networking, accessibility, error handling, security, testing and maintainability.

A useful architecture separates UI/presentation from domain rules and data access. Flutter's current architecture guidance explicitly emphasizes separation of concerns, layered architecture, single source of truth, unidirectional data flow and testability. ■cite■turn0search7■turn0search24■

Core Concepts — Detailed Breakdown

- Requirements: define users, actions, constraints and success criteria before coding.
- Architecture: separate presentation, state/domain logic and repositories/services.
- Persistence: choose local storage vs remote database based on data requirements.
- State: model loading, success, empty and error states explicitly.
- Validation: validate at the UI for fast feedback and again at the trusted backend boundary.
- Accessibility: labels, contrast, touch targets, focus order and meaningful feedback.
- Testing: unit-test business rules; integration-test critical flows.
- Observability: record useful non-sensitive logs and crash information.
- Security: protect credentials, enforce authorization and minimize stored sensitive data.

What You Should Be Able To Do

- Write a short requirements document.
- Draw the data flow from UI → state/domain → service → backend.
- Implement at least one unit test and one end-to-end critical flow.
- Test slow network and invalid input.
- Create a README with architecture and trade-offs.

Hands-on Project / Practice

- Build a campus marketplace/event/expense app with authentication, persistent data, search/filter, validation, loading/error states and a polished README. Include screenshots and an architecture diagram.

Recommended References

- Flutter App Architecture — <https://docs.flutter.dev/app-architecture>
- Flutter State Management — <https://docs.flutter.dev/data-and-backend/state-mgmt/intro>
- Firebase Documentation — <https://firebase.google.com/docs>
- React Native Documentation — <https://reactnative.dev/docs/getting-started>

Note: These notes are designed to teach the concepts and provide a practical learning path. Always prefer the linked official documentation for current APIs, SDK versions and platform-specific release requirements.